PlaceIQ

A College Placement Management System

with database-enforced placement rules, analytics and a placement-probability model

Database Management Systems: Digital Assignment 3 (Project Report)

[Course Code] – [Course Name]

Submitted by

Name	Register Number
[Student Name 1]	[Register Number 1]
[Student Name 2]	[Register Number 2]

Under the guidance of

[Faculty Name]
[Designation, School / Department]

[Institution Name]
[Date of Submission]

Abstract

Campus placement in most colleges is still coordinated through spreadsheets, forms and e-mail, so the rules that matter most (who is eligible for a drive, whether a student may hold two accepted offers, which status changes are legal) live in people's heads or in scattered application code. This project, PlaceIQ, is a placement management system whose central design decision is to enforce those rules **inside the database**. A PostgreSQL 16 schema of 13 tables in third normal form carries six triggers, two stored functions, one stored procedure, six views (three of them built on window functions) and a materialized view. Invalid writes are refused by the database itself with custom error codes (PL001–PL007), whether they come from the web application or from a raw SQL session. A Flask backend written with plain SQL (psycopg2, no ORM) exposes 36 REST endpoints, runs offer acceptance as an explicit transaction, and serves a React front end with a student portal, an admin console and an analytics dashboard. A logistic-regression model estimates each student's placement probability. The schema was also ported to Oracle 21c and run end to end in SQL*Plus.

On a synthetic dataset of 180 students, 25 drives and 517 applications, an end-to-end smoke test passed 41 of 41 checks, an automated check of the API documentation passed 161 of 161, and a browser walkthrough of the user interface passed 28 of 28. Every business rule was shown to be refused at the database level, including two simultaneous offer acceptances, which the trigger's row lock serialized. On a 250,000-application benchmark database the indexes cut query time by between 1.8× and 81×. The placement model reached a test accuracy of 0.750 and a ROC-AUC of 0.774, but on a test set of only 36 synthetic students, so it demonstrates the pipeline rather than real predictive power.

Keywords: relational database, triggers, stored procedures, transactions, window functions, materialized views, indexing, placement management, logistic regression.

Table of Contents

Abstract 1
1 Introduction 3
1.1 Literature Review 3
1.2 Problem Statement 4
1.3 Objectives 4
1.4 Scope 4
2 Modules (Implementation and Experimental Setup) 5
2.1 System Architecture 5
2.2 Database Design 5
2.3 Database Logic: Triggers, Routines and Views 7
2.4 Backend API Module 8
2.5 User Interface Module 9
2.6 Placement-Probability Module 10
2.7 Oracle Port 10
2.8 Experimental Setup 10
3 Results and Discussion 12
3.1 Functional Verification 12
3.2 Rules Enforced by the Database 12
3.3 Transactions and Concurrency 13
3.4 Placement Analytics 13
3.5 Effect of the Indexes 16
3.6 Placement-Probability Model 16
4 Conclusion and Future Work 18
References 19
Appendix A Code Listings 21
Appendix B Screenshots 25
Appendix C Evidence Files 30

1 Introduction

Every placement season a college's training and placement office (TPO) runs dozens of recruitment drives. Each drive has its own eligibility rules (a minimum CGPA, a maximum number of active backlogs, the branches it is open to), and each applicant moves through a pipeline of shortlisting, interview rounds and offers. The office has to answer two kinds of question at once: operational ones ("who is eligible for this drive?", "which applicants are still pending?") and analytical ones ("what share of each department has been placed?", "which recruiters made the most offers?").

PlaceIQ is a web-based system for this workflow, built for the DA-3 project of the Database Management Systems course. Its emphasis is the database: the schema, the rules it enforces, the queries behind the analytics, and the transactions that keep related tables consistent.

1.1 Literature Review

The relational model and normalization. Codd's relational model [1] proposed representing data as relations and introduced a normal form for them, separating how data is logically organized from how it is stored. Normalization to third normal form, as presented in standard texts such as Silberschatz, Korth and Sudarshan [3], removes partial and transitive dependencies so that each fact is stored once. PlaceIQ's schema follows this: department and company details are kept in their own tables instead of being copied onto every student and drive row.

Transactions. Härder and Reuter [2] set out the terminology of transaction-oriented recovery and the properties now known as ACID: atomicity, consistency, isolation and durability. Accepting an offer in PlaceIQ updates three tables, and atomicity is what guarantees they never disagree. PostgreSQL's documentation describes how its transactions [10] and row-level locks [11] provide these guarantees in practice, and how trigger functions [8] can enforce rules on every write.

Existing placement management systems. Santhosh Kumar and Srividhya [7] describe a web application that automates registration, record updates, searching and notifications for an engineering college's training and placement unit. Systems of this kind digitize the office's records, but the rules are typically checked in application code, which a direct database edit or a second client can bypass.

Predicting placement. Several studies predict whether a student will be placed from academic records. Shukla et al. [5] built a logistic-regression model that estimates a student's placement probability from historical data with known outcomes. Kumar et al. [6] compared several machine-learning methods on students' academic performance and found Random Forest the most effective. PlaceIQ uses logistic regression, implemented with scikit-learn [4], because its coefficients can be read directly and explained to a student. The prediction is a supporting feature here, not the focus.

Gap addressed. The reviewed placement systems treat the database mainly as storage. PlaceIQ moves the placement rules into the database (triggers and stored routines), adds analytics

computed with window functions and a materialized view, and measures the effect of its indexes instead of assuming it.

1.2 Problem Statement

Design and implement a placement management system in which the placement office's rules are guaranteed by the database, so that no client (the web application, an admin's SQL session or a future integration) can create an inconsistent state. Examples of such states are an ineligible application, two accepted offers for one student, an offer for a rejected applicant, or an application moving backwards in the pipeline. The system must also give students and the placement office the operational and analytical views they need, and its design choices (normalization, indexing, transactions) must be justified with measured evidence.

1.3 Objectives

- Design a normalized (3NF) relational schema for students, departments, skills, companies, drives, applications, interview rounds and offers, with an audit trail of status changes.
- Enforce the placement rules in the database with triggers, a stored function for eligibility and a stored procedure for closing a drive, each failing with a distinct error code.
- Implement offer acceptance and decline as explicit transactions, and show that they are atomic and safe under concurrent requests.
- Provide analytics through views, window functions and a materialized view, and present them in a web dashboard.
- Measure the effect of the indexes with EXPLAIN ANALYZE on a realistically sized dataset.
- Integrate a small, explainable placement-probability model and report its accuracy honestly.
- Port the database layer to Oracle and run it end to end in SQL*Plus.

1.4 Scope

In scope:

- Two roles: a student (register, view eligible drives, apply, track applications, accept or decline offers) and the placement officer (manage drives, move applicants through the pipeline, record interview rounds and offers, close drives, view analytics).
- The PostgreSQL schema, the Flask REST API, the React user interface, the ML service, and the Oracle port of the schema.

Out of scope:

- Real student data and any connection to an institution's academic systems. All data is synthetic, and CGPA is entered by the student at sign-up.
- E-mail or SMS notifications, document uploads (resumes, offer letters) and recruiter logins.
- Running the backend against Oracle: the Oracle port covers the database layer only.

2 Modules (Implementation and Experimental Setup)

2.1 System Architecture

PlaceIQ is a three-tier web application with a separate prediction service (Table 1). The browser talks only to the backend. The backend talks to PostgreSQL with plain SQL through a connection pool and calls the ML service over HTTP when a student asks for their placement probability.

Table 1: Components and technologies

Tier	Technology	Responsibility
Presentation	React 19, React Router 7, Vite 8	Student portal, placement-office console, analytics dashboard (charts drawn as SVG)
Application	Python, Flask 3.0, psycopg2 (no ORM), PyJWT, bcrypt	36 REST endpoints, authentication, role checks, offer transactions
Data	PostgreSQL 16 (Docker)	13 tables, 6 triggers, 2 functions, 1 procedure, 6 views, 1 materialized view
Prediction	Flask, scikit-learn, pandas	Trains the model from the database and serves POST /predict
Oracle port	Oracle Database 21c XE, SQL*Plus	The same schema and rules in PL/SQL

The backend deliberately uses raw SQL rather than an object-relational mapper. Every query the system runs is visible in the source, which matters for a database course. It also means the transaction boundaries are explicit `commit()` and `rollback()` calls rather than behaviour hidden inside a framework.

2.2 Database Design

Figure 1 shows the entity–relationship diagram of the 13 tables, rendered from the Mermaid source in `docs/er-diagram.md`. The central entity is `application`, which resolves the many-to-many relationship between students and drives and carries its own attributes (status and application time). Two further junction tables resolve students↔skills and drives↔eligible departments. A drive with no rows in `drive_eligible_dept` is open to every department.

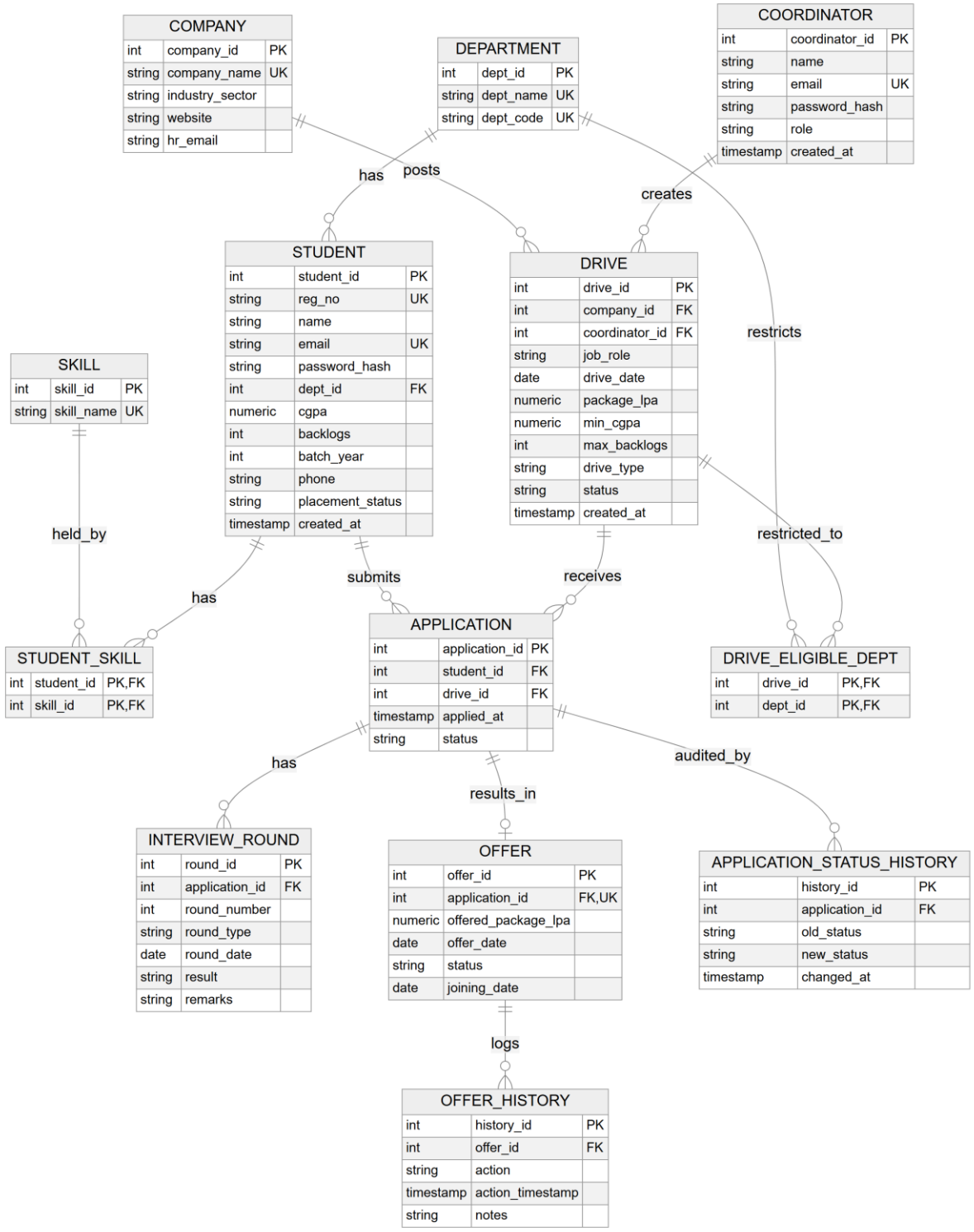


Figure 1: Entity–relationship diagram (13 tables)

Table 2: Tables

Table	Purpose	Key constraints
department	The six branches	UNIQUE name and code
coordinator	Placement-office users (TPO / coordinator)	UNIQUE e-mail; CHECK role
student	Profile, CGPA, backlogs, placement status	UNIQUE reg. no. and e-mail; CHECK $0 \le$ CGPA ≤ 10; FK department

Table	Purpose	Key constraints
skill, student_skill	Skills and who holds them (M:N)	Composite PK (student, skill)
company	Recruiters	UNIQUE name
drive	One recruitment posting with its cutoffs	FK company, coordinator; CHECK status, type
drive_eligible_dept	Branches a drive is open to (M:N)	Composite PK (drive, department)
application	A student's application to a drive	UNIQUE (student, drive); CHECK status
interview_round	Rounds of one application	UNIQUE (application, round no.)
offer	At most one offer per application	UNIQUE application; CHECK status
offer_history	Audit log of offer events	CHECK action
application_status_history	Audit log of status changes (written by a trigger)	FK application

All 13 tables are in third normal form: every non-key attribute depends on the whole key and on nothing else. The one deliberate exception is `student.placement_status`, which could be derived from the offers. It is stored because it is read on almost every page, and a trigger keeps it in step with the offer table, so it cannot drift. The full functional-dependency analysis is in `docs/normalization.md`, and every column and constraint, read from the system catalog, is in `docs/evidence/schema-summary.md`.

2.3 Database Logic: Triggers, Routines and Views

The placement rules are implemented as BEFORE triggers that inspect the row being written and raise an error with a custom SQLSTATE code if it breaks a rule (Table 3). Because the check runs inside the database, it applies to every client. The backend translates each code into an HTTP status (`backend/utils/db_errors.py`), so the user sees a meaningful message rather than a generic failure.

Table 3: Database-enforced rules

Code	Trigger / routine	Rule	HTTP
PL001	trg_application_before_insert	A student may apply only if eligible (CGPA, backlogs, branch)	403
PL002	trg_application_before_insert	Applications are accepted only while a drive is Upcoming or Ongoing	409
PL003	trg_offer_before_accept	A student may hold at most one Accepted offer	409
PL004	sp_close_drive	The drive to close must exist	404
PL005	sp_close_drive	A drive can be closed only once	409
PL006	trg_offer_requires_selected	An offer can be recorded only for a Selected application	409
PL007	trg_application_status_transition	Applications move only forward through the pipeline	409

The legal status transitions form a small state machine: Applied → Shortlisted / Interview_Scheduled / Rejected; Shortlisted → Interview_Scheduled / Selected / Rejected; Interview_Scheduled → Selected / Rejected; Selected → Offer_Made / Rejected. Offer_Made and Rejected are final. A CHECK constraint cannot express this rule because it sees only the new value. The trigger compares `OLD.status` with `NEW.status`.

Two AFTER triggers maintain derived data. `trg_offer_after_insert` marks a student Placed when an offer is recorded. `trg_application_status_audit` writes one

application_status_history row for every status change, so no code path can change a status without leaving a record.

Rule PL003 and concurrency. If two requests accept two different offers of the same student at the same moment, neither can see the other's uncommitted change, and a naive count of Accepted offers would let both through. The trigger therefore first locks the student's row with SELECT ... FOR UPDATE, so the second request waits until the first commits and then sees its result (Section 3.3).

Stored routines. fn_check_eligibility(student, drive) returns whether a student meets a drive's rules. It is the single definition of eligibility, used by the application trigger and by fn_get_eligible_drives(student), which returns every open drive a student qualifies for. The procedure sp_close_drive(drive, INOUT rejected_count) rejects every still-pending applicant and marks the drive Completed as one unit of work.

Views. Three views serve the dashboards (placed_students_view, company_wise_stats_view, department_stats_view). Three more use window functions: dept_offer_ranking_view ranks offers within each department with RANK(); placement_timeline_view computes a running total with SUM(...) OVER and the month-on-month change with LAG(); student_cgpa_standing_view uses DENSE_RANK() and PERCENT_RANK() for each student's position within their department [9]. The materialized view mv_company_dept_placements stores offers per company and department. It is refreshed on demand with REFRESH MATERIALIZED VIEW CONCURRENTLY, which needs a unique index and lets readers continue during the refresh [12], and the dashboard shows when it was last refreshed.

Indexes. Besides the indexes created by primary-key and unique constraints, five indexes support the frequent lookups: student(dept_id), student(cgpa), application(status), application(drive_id) and drive(company_id) [13]. Their effect is measured in Section 3.5.

2.4 Backend API Module

The Flask backend is organized in blueprints (authentication, students, drives, applications, offers, analytics) and exposes 36 endpoints, each documented in docs/API.md. Users authenticate with e-mail and a bcrypt-hashed password and receive a JSON Web Token [14] that carries their role. Every endpoint checks the role, and students can read and change only their own records. Database access goes through a psycopg2 connection pool [15], with parameterized queries throughout.

The offer transaction. Accepting an offer must update three things together: the offer becomes Accepted with a joining date, the student is marked Placed, and an Accepted row is appended to offer_history. offer_service.accept_offer runs the three statements on one connection inside try, calls commit() only after all three succeed, and calls rollback() if any statement fails, including when a trigger refuses it. Declining works the same way, and it also sets the student back to Not Placed if no other live offer remains. The full function is listed in Appendix A.

2.5 User Interface Module

The React front end has three areas. Students see an overview (profile, CGPA standing in their department, placement probability), their eligible drives, and their applications. The application detail shows interview rounds, the offer and the status history written by the audit trigger (Figure 2). The placement office manages drives and applicants (Figure 3) and sees an analytics dashboard (Section 3.4). Every chart also has a table view, so no value is available only by hovering.

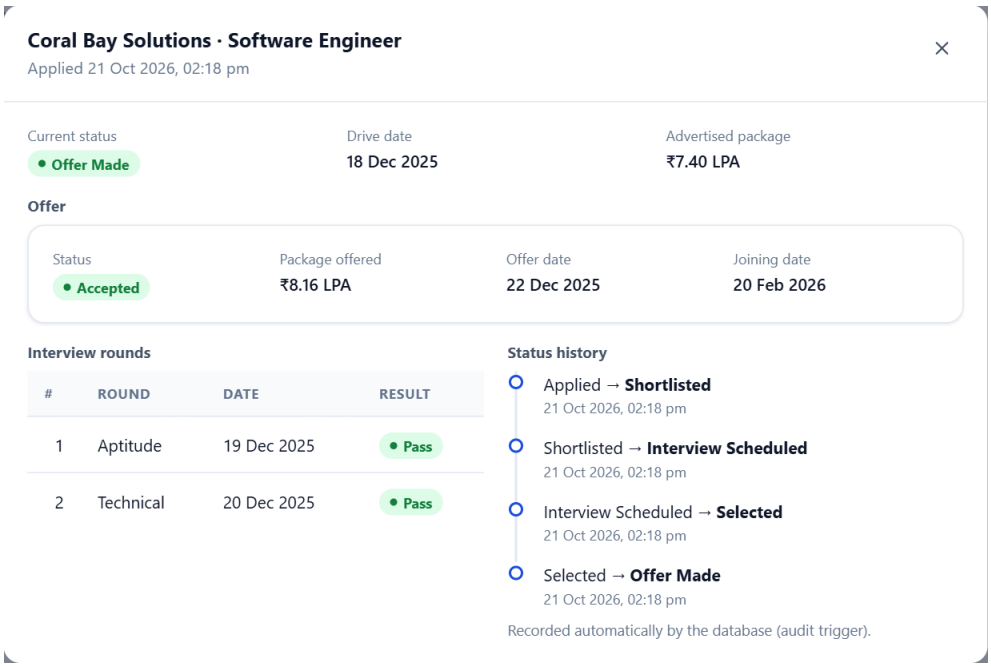


Figure 2: Application detail: offer, interview rounds and the status history recorded by the audit trigger

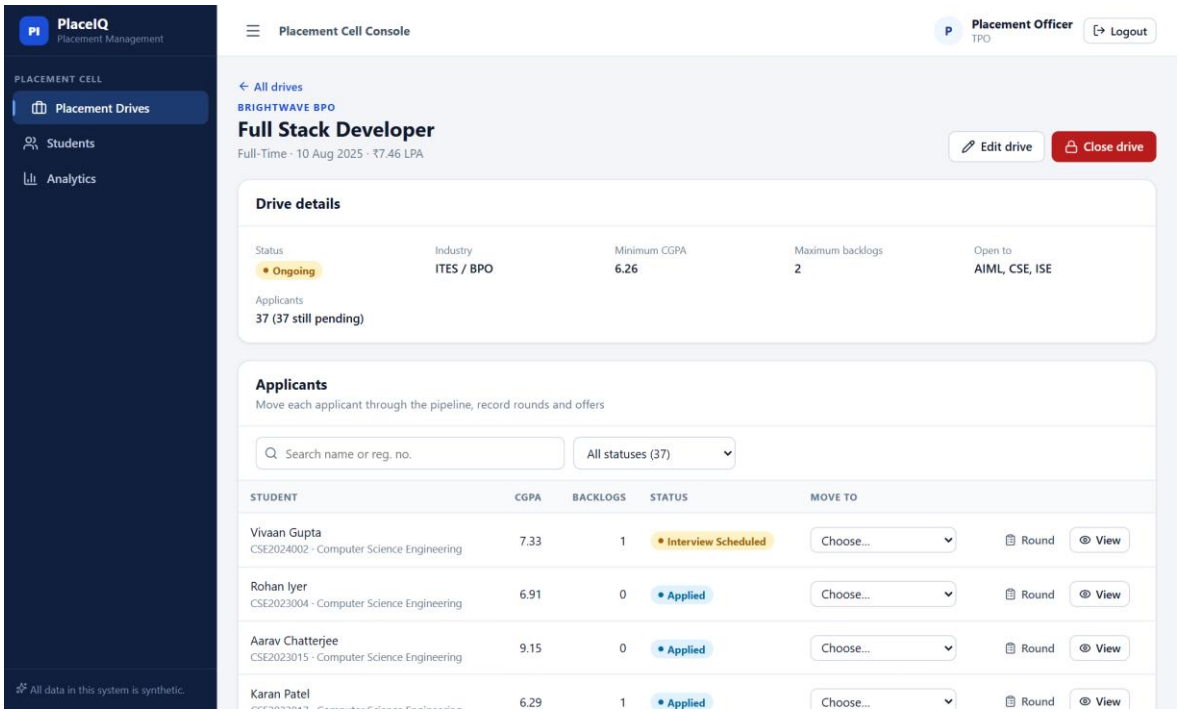


Figure 3: Placement office: a drive's details and its applicants

2.6 Placement-Probability Module

A separate service trains a scikit-learn [4] pipeline: standardized numeric features, a one-hot encoded department, and a logistic-regression classifier. The features are CGPA, number of active backlogs, number of skills (counted with a join on `student_skill`) and department. The label is `placement_status`, which the database trigger sets. The data is split 80/20, stratified by label, with a fixed random seed. The trained model is saved to disk and served by `POST /predict`. The backend calls it for the logged-in student and returns HTTP 503 if the service is unavailable.

2.7 Oracle Port

The schema was ported to Oracle Database 21c Express Edition (`db/schema_oracle.sql`) and run end to end in a single SQL*Plus session (`db/oracle_full_run.sql`). The main differences are:

- Identity columns are implemented with sequences and BEFORE INSERT triggers.
- Rule violations are raised with `RAISE_APPLICATION_ERROR` (−20001 to −20007) instead of custom SQLSTATEs.
- The materialized view is built `REFRESH COMPLETE ON DEMAND`.
- Identifiers are shortened to Oracle's 30-character limit.
- Rule PL003 is a COMPOUND trigger [16]. A row-level trigger on `offer` cannot query `offer` itself (ORA-04091, "table is mutating"), so the compound trigger collects the affected rows and checks them after the statement.

The differences are documented line by line in `docs/oracle-differences.md`.

2.8 Experimental Setup

Table 4: Environment

Item	Value
Machine	12th Gen Intel Core i7-12700H, 15.6 GB RAM, Windows 11
Database	PostgreSQL 16.15 in Docker (postgres:16 image)
Oracle	Oracle Database 21c XE (gvenzl/oracle-xe:21-slim) with SQL*Plus 21.3
Backend / ML	Python 3.14, Flask 3.0.3, psycopg2 2.9, scikit-learn 1.5+
Front end	React 19, Vite 8, tested in headless Chromium (Playwright)

Dataset. No real student data was used. `db/seed.py` generates a deterministic synthetic dataset: 180 students (30 in each of six departments), 18 companies, 25 drives, 517 applications and 177 offers, after which 95 students (52.8%) are Placed. The seed moves every application through legal status steps, so the audit trigger logged 1,177 status changes. Offers are made with a probability that rises with CGPA margin and skill count and falls with backlogs, so the dataset has a learnable signal.

Tests and measurements.

- **Smoke test** (`tests/smoke_test.py`): 41 checks through the API and directly against the database. It runs on a throwaway copy of the database, built and dropped by `tests/run_smoke_test.sh`.

- **API contract check** (`tests/api_contract_check.py`): parses `docs/API.md` and calls every documented endpoint, comparing status codes and response fields.
- **UI walkthrough** (`tests/ui_walkthrough.js`): Playwright drives the real interface through each flow.
- **Index benchmark** (`db/demos/index_benchmark.py`): a separate database of 50,000 students, 2,000 drives and 250,000 applications. Each query's median time over 7 warm runs is measured with and without the indexes.
- **SQL evidence**: every query, trigger and transaction was run in psql. Anything that changed data ran inside a transaction that was rolled back. The unedited output is in `docs/evidence/`.

3 Results and Discussion

All figures in this chapter are copied from the evidence files in `docs/evidence/` (listed in `docs/evidence/INDEX.md`). Each file holds the unedited output of the run it describes.

3.1 Functional Verification

Table 5: Verification results

Check	Result	Evidence
End-to-end smoke test (API and database)	41 / 41 passed	smoke-test.md
API documentation vs. running backend (36 endpoints)	161 / 161 passed	raw/api-contract-check.txt
Browser walkthrough of the UI	28 / 28 passed	raw/ui-walkthrough.txt
Oracle schema in SQL*Plus	82 objects, all VALID; no compilation errors	sqlplus-transcript.txt
Oracle rule triggers	All 6 intended ORA-200xx errors raised, no others	sqlplus-transcript.txt

3.2 Rules Enforced by the Database

Each rule was tested by sending a violating statement straight to PostgreSQL, bypassing the backend (`triggers.md`). Table 6 lists what was run and what the database answered. Each refused write left no row behind. For example, after the PL006 attempt, application 1 still had 0 offers.

Table 6: Rule violations sent directly to the database

Statement (summary)	Database response	Code
Student 8 (CGPA below cutoff) applies to drive 1	Student 8 is not eligible for drive 1	PL001
A student applies to drive 2, which is Completed	Drive 2 is Completed and no longer accepts applications	PL002
Student 18, who already accepted an offer, accepts offer 70	Student 18 has already accepted another offer	PL003
Insert an offer for application 1 (Rejected)	Application 1 is Rejected; an offer can only be recorded for a Selected application	PL006
Move application 1 from Rejected to Applied	Application 1 cannot move from Rejected to Applied	PL007
Move application 2 from Offer_Made to Shortlisted	Application 2 cannot move from Offer_Made to Shortlisted	PL007

The function `fn_check_eligibility` was checked against an independently computed reason for four real student–drive pairs (`queries.md`). It returned true for the pair meeting every rule. It returned false for a CGPA of 7.99 against a cutoff of 8.25, for 2 backlogs against a maximum of 1, and for a CSE student applying to a drive open only to AIML and ECE. Calling `sp_close_drive` on drive 24 (Brightwave BPO, Full Stack Developer) rejected its 37 pending applicants (27 Applied, 5 Shortlisted, 5 Interview_Scheduled) and marked it Completed, and the audit trigger logged all 37 changes. The demonstration was then rolled back, and the drive was again Ongoing.

3.3 Transactions and Concurrency

The offer transactions were run through the real backend with the database state read before and after each call (`transaction.md`):

- **Accept**: offer 178 went from Pending to Accepted with a joining date of 2026-12-20, and the history went from [Created] to [Created, Accepted] in one request.
- **Decline** of a student's only offer: the offer became Declined and `placement_status` returned from Placed to Not Placed, together with the history row.
- **Refused acceptance** (PL003): the backend answered HTTP 409, and the before and after states were identical. The transaction left nothing behind.
- **Two simultaneous acceptances** from two database connections: connection B was still blocked on the student-row lock 1.5 s later. When A committed, B was immediately refused with PL003. Offer 180 ended Accepted and offer 181 stayed Pending, exactly one acceptance.

At the SQL level (`transaction-sql.md`), the same three statements were run by hand and then rolled back, and all three changes disappeared together. In a second run a CHECK-constraint violation in the third statement aborted the transaction: PostgreSQL refused every following statement, the final COMMIT became a rollback, and the first two updates were undone. This is atomicity in practice [2], [10].

3.4 Placement Analytics

Table 7 is the output of `department_stats_view` on the seeded data. Electrical & Electronics has the highest placement rate (60.0%) and Computer Science the lowest (43.3%). Because the data is synthetic, these figures describe the generator's choices, not any real college.

Table 7: Placement by department (department_stats_view)

Department	Students	Placed	Placed %
Electrical & Electronics	30	18	60.00
Artificial Intelligence & ML	30	17	56.67
Information Science Engineering	30	16	53.33
Electronics & Communication	30	16	53.33
Mechanical Engineering	30	15	50.00
Computer Science Engineering	30	13	43.33

Table 8: Top recruiters by offers (company_wise_stats_view)

Company	Offers	Average package (LPA)
Coral Bay Solutions	80	6.89
Everstone Infra	41	7.14
Solstice Retail Tech	24	5.51
Bluepeak Software	8	14.62
Meridian Consulting	8	10.14

Offers per month (from `placement_timeline_view`) were 3, 7, 1, 36, 114, 0 and 16 for August 2025 to February 2026, a running total of 177 (Figure 5). Capturing this screen exposed a defect. The first version of the view only had rows for months with offers, so `LAG()` compared February with December, because January had no offers, and reported a fall of 98. The view

now generates every month between the first and last offer (`generate_series` in PostgreSQL, `ADD_MONTHS` with `CONNECT BY LEVEL` in Oracle) and left-joins the counts, so January appears as 0 and February's change is +16. The fix was re-verified on both databases. It shows a general pitfall of `LAG()`: it reads the previous row, not the previous calendar period.

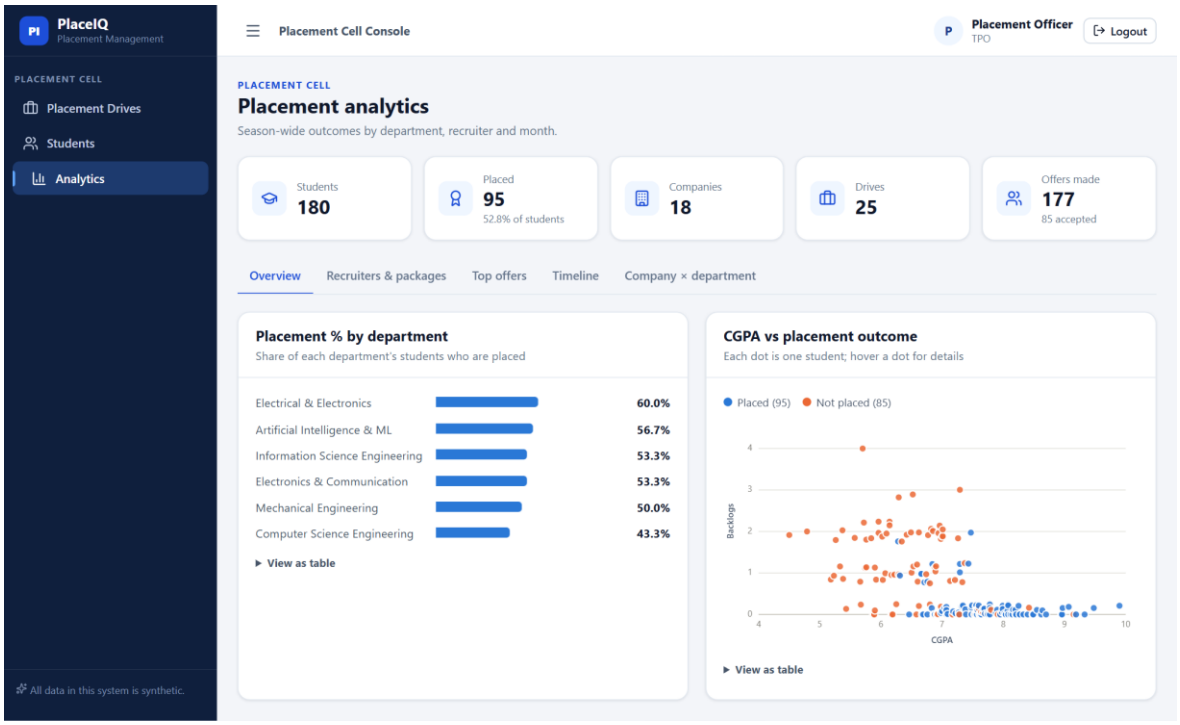


Figure 4: Analytics: placement % by department and CGPA against placement outcome

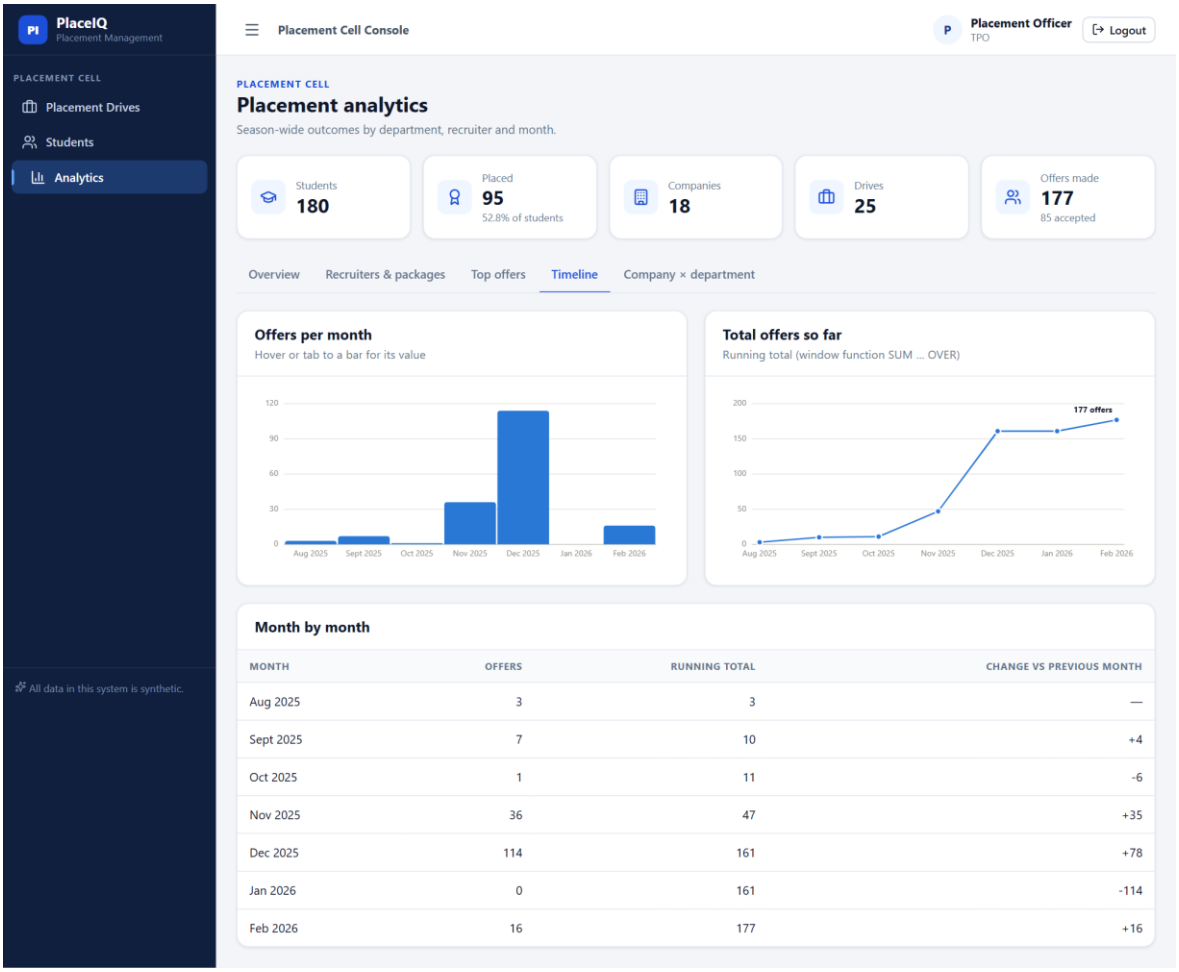


Figure 5: Offers per month and running total, from the window-function view

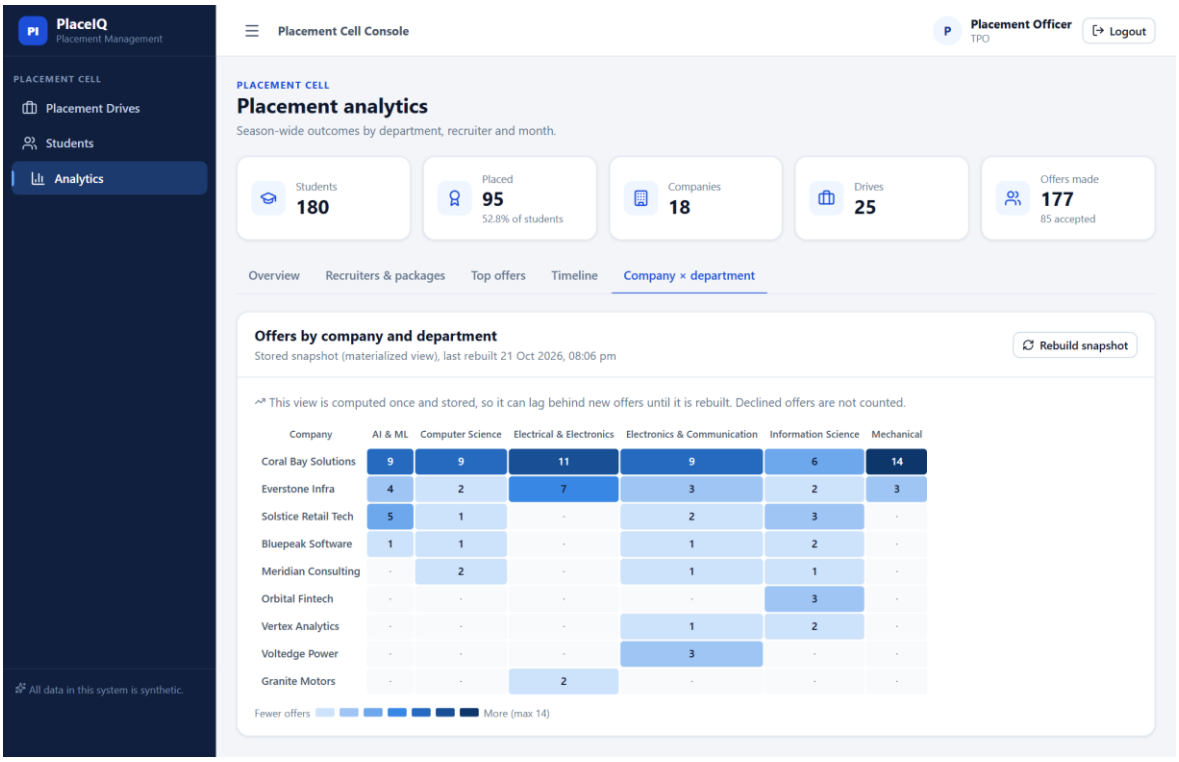


Figure 6: Offers by company and department, read from the materialized view

3.5 Effect of the Indexes

Table 9 gives the median execution time of five of the application's lookups on the 250,000-application benchmark database, with and without the indexes (`index-benchmark.md`, which also contains every query plan).

Table 9: Index benchmark (median of 7 warm runs)

Query	Index used	With	Without	Speed-up
Applicants for one drive	idx_application_drive	0.13 ms	10.31 ms	81.2×
CSE students with CGPA ≥ 9.7	idx_student_cgpa + idx_student_dept	0.62 ms	6.15 ms	9.9×
Count of students in one department	idx_student_dept	2.08 ms	7.20 ms	3.5×
Count of applications in one status	idx_application_status	4.56 ms	13.23 ms	2.9×
Drives of one company	idx_drive_company	0.07 ms	0.12 ms	1.8×

The gain depends on selectivity. Listing one drive's applicants touches 125 of 250,000 rows. Through the index PostgreSQL read 127 buffer pages instead of scanning all 1,966, an 81× speed-up. For the CGPA and department query the planner combined two indexes with a `BitmapAnd` before reading the table. A filter that matches a sixth of the students (one department) or 3% of the applications (one status) still has to visit most pages, so the gain is smaller. The `drive.company_id` index barely helps (0.07 vs 0.12 ms) because the whole `drive` table is 27 pages. It is kept for the drive-to-company join, but the measurement does not justify it on speed alone. On the real 180-student dataset none of the indexes matters, which is why the benchmark uses a separate, larger database.

3.6 Placement-Probability Model

Table 10: Model results on the held-out test set

Measure	Value
Training / test rows	144 / 36 (test: 19 placed, 17 not placed)
Training accuracy	0.875
Test accuracy	0.750 (27 of 36 correct)
Test ROC-AUC	0.774
Precision / recall for 'placed'	0.750 (15 of 20) / 0.789 (15 of 19)
Confusion matrix (actual rows × predicted)	Not placed: 12 correct, 5 wrong; Placed: 15 correct, 4 wrong

The coefficients point the expected way. CGPA has the largest positive weight (+1.496 log-odds per standard deviation), backlogs the largest negative (−1.136), and the skill count is positive (+0.819). Department effects are smaller. Figure 7 shows the estimate as a student sees it.

These results must be read with care. The test set has only 36 students, so one prediction changes accuracy by about 2.8 percentage points, and another random split could easily give 0.70 or 0.80. The data is synthetic, and the generator itself makes CGPA, backlogs and skills predictive, so the model largely recovers rules that were built in. The result shows that the pipeline works end to end (database, features, model, API, interface). It does not show that the model would predict real placements. For comparison, Kumar et al. [6] report Random Forest

as the most effective of the methods they compared on real student records. Choosing a model on real data would need a comparison of that kind.

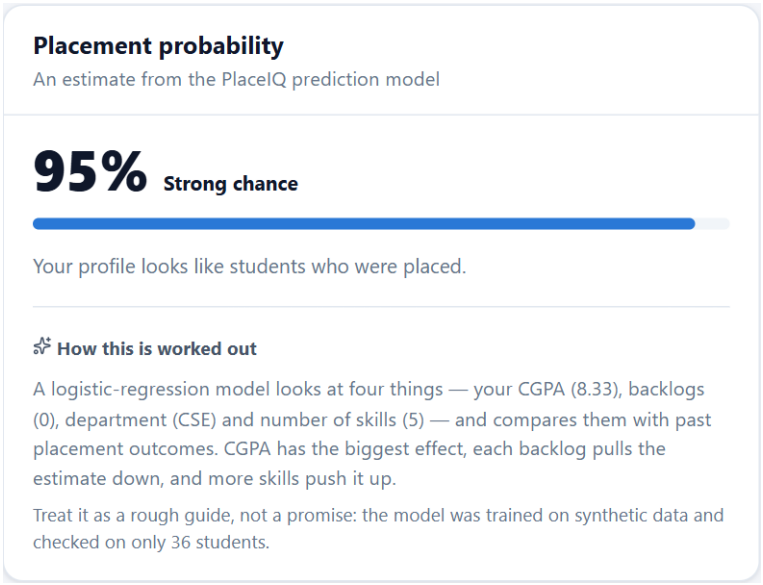


Figure 7: The placement-probability card on the student dashboard

4 Conclusion and Future Work

PlaceIQ shows that a placement office's rules can live in the database rather than in application code. Eligibility, the one-accepted-offer rule, offers only for selected applicants and forward-only status changes are all enforced by triggers, and each was shown to refuse a violating write sent directly to PostgreSQL. The offer transactions are atomic and remain correct under concurrent requests because the trigger locks the student's row. Window functions and a materialized view supply the analytics. The indexes were measured rather than assumed: up to 81× faster for selective lookups, and negligible on a tiny table. The same design runs on Oracle 21c with all 82 objects valid.

Limitations

- **Synthetic data.** All students, companies and outcomes are generated. The analytics describe the generator, not a real college.
- **Self-reported CGPA.** Students enter their own CGPA and backlogs at sign-up, and eligibility depends on them. A real deployment would need these values verified against the institution's records through an official integration, not entered by the student.
- **Small ML evaluation.** The model was tested on 36 synthetic students with a single split. Its accuracy is a rough figure, and its strength largely reflects how the data was generated.
- **Oracle port covers the database layer only.** The backend runs only against PostgreSQL. Mapping Oracle's −20001 to −20007 errors in the backend and the Oracle date arithmetic in the offer service were not built. On Oracle, the concurrent-acceptance behaviour rests on documented locking and was not demonstrated, because the demo is a single SQL*Plus session. Oracle has no `CONCURRENTLY` refresh option.
- **Seeded timestamps.** The seed leaves application and status-history timestamps at the time it ran, so in the demo data an application can appear to be dated after its drive.
- **Placement policy.** A placed student may keep applying to other drives. Many colleges restrict this, and the rule would be one more trigger.

Future Work

- Verify academic records through an official institutional data feed instead of self-reported values.
- Evaluate the model on real, consented and anonymized placement records with cross-validation, and compare it with other classifiers such as Random Forest [6].
- Add recruiter accounts, e-mail notifications and document uploads (resumes, offer letters).
- Make the placement policy configurable (for example, how many offers a placed student may hold) and enforce it with a trigger.
- Complete an Oracle-backed deployment of the backend and demonstrate concurrent acceptance there.
- Schedule the materialized-view refresh, or refresh it when offers change, instead of on demand only.

References

[1] E. F. Codd, "A relational model of data for large shared data banks," Communications of the ACM, vol. 13, no. 6, pp. 377–387, 1970, doi: 10.1145/362384.362685.

[2] T. Härder and A. Reuter, "Principles of transaction-oriented database recovery," ACM Computing Surveys, vol. 15, no. 4, pp. 287–317, 1983, doi: 10.1145/289.291.

[3] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York: McGraw-Hill, 2019. [Online]. Available: https://www.db-book.com/

[4] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://jmlr.org/papers/v12/pedregosa11a.html

[5] M. K. Shukla, P. Rambade, J. Torasakar, R. Prabhu, and D. Maste, "Students placement prediction model using logistic regression," International Journal of Engineering Research & Technology (IJERT), ICIATE – 2017, vol. 5, no. 01, 2017. [Online]. Available: https://www.ijert.org/students-placement-prediction-model-using-logistic-regression

[6] M. Kumar, N. Walia, S. Bansal, G. Kumar, and K. Cengiz, "Predicting college students' placements based on academic performance using machine learning approaches," International Journal of Modern Education and Computer Science, vol. 15, no. 6, pp. 1–13, 2023, doi: 10.5815/ijmecs.2023.06.01.

[7] H. Santhosh Kumar and V. R. Srividhya, "Online training and placement management system," International Journal of Engineering Research & Technology (IJERT), ICACT – 2016, vol. 4, no. 22, 2016. [Online]. Available: https://www.ijert.org/online-training-and-placement-management-system

[8] The PostgreSQL Global Development Group, "Trigger functions," PostgreSQL 16 Documentation, §43.10. [Online]. Available: https://www.postgresql.org/docs/16/plpgsql-trigger.html

[9] The PostgreSQL Global Development Group, "Window functions," PostgreSQL 16 Documentation, §3.5. [Online]. Available: https://www.postgresql.org/docs/16/tutorial-window.html

[10] The PostgreSQL Global Development Group, "Transactions," PostgreSQL 16 Documentation, §3.4. [Online]. Available: https://www.postgresql.org/docs/16/tutorial-transactions.html

[11] The PostgreSQL Global Development Group, "Explicit locking," PostgreSQL 16 Documentation, §13.3. [Online]. Available: https://www.postgresql.org/docs/16/explicit-locking.html

[12] The PostgreSQL Global Development Group, "Materialized views," PostgreSQL 16 Documentation, §41.3. [Online]. Available: https://www.postgresql.org/docs/16/rules-materializedviews.html

[13] The PostgreSQL Global Development Group, "Using EXPLAIN," PostgreSQL 16 Documentation, §14.1. [Online]. Available: https://www.postgresql.org/docs/16/using-explain.html

[14] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519

[15] Psycopg – PostgreSQL database adapter for Python, documentation. [Online]. Available: https://www.psycopg.org/docs/

[16] Oracle, "PL/SQL triggers," Oracle Database PL/SQL Language Reference, 21c. [Online]. Available: https://docs.oracle.com/en/database/oracle/oracle-database/21/lnpls/plsql-triggers.html

Appendix A Code Listings

The listings below are read from the source files when this report is generated, with their real line numbers, so they match the submitted code exactly.

A.1 Rule PL003: one accepted offer per student, with the row lock

`db/schema.sql`, lines 513–546.

```
513  CREATE OR REPLACE FUNCTION trg_fn_offer_single_acceptance()
514  RETURNS TRIGGER AS $$
515  DECLARE
516      v_student_id student.student_id%TYPE;
517  BEGIN
518      IF NEW.status <> 'Accepted' THEN
519          RETURN NEW;
520      END IF;
521
522      SELECT student_id INTO v_student_id
523        FROM application WHERE application_id = NEW.application_id;
524
525      PERFORM 1 FROM student WHERE student_id = v_student_id FOR UPDATE;
526
527      IF EXISTS (
528          SELECT 1
529            FROM offer o
530            JOIN application a ON a.application_id = o.application_id
531           WHERE a.student_id = v_student_id
532             AND o.status = 'Accepted'
533             AND o.offer_id IS DISTINCT FROM NEW.offer_id
534      ) THEN
535          RAISE EXCEPTION 'Student % has already accepted another offer', v_student_id
536              USING ERRCODE = 'PL003';
537      END IF;
538
539      RETURN NEW;
540  END;
541  $$ LANGUAGE plpgsql;
542
543  CREATE TRIGGER trg_offer_before_accept
544  BEFORE INSERT OR UPDATE OF status ON offer
545  FOR EACH ROW
546  EXECUTE FUNCTION trg_fn_offer_single_acceptance();
```

A.2 Rule PL007: the status-transition state machine

`db/schema.sql`, lines 595–622.

```
595  CREATE OR REPLACE FUNCTION trg_fn_application_status_transition()
596  RETURNS TRIGGER AS $$
597  DECLARE
598      v_allowed BOOLEAN;
599  BEGIN
600      v_allowed := CASE OLD.status
601          WHEN 'Applied'             THEN NEW.status IN ('Shortlisted', 'Interview_Scheduled', 'Rejected')
602          WHEN 'Shortlisted'         THEN NEW.status IN ('Interview_Scheduled', 'Selected', 'Rejected')
603          WHEN 'Interview_Scheduled' THEN NEW.status IN ('Selected', 'Rejected')
604          WHEN 'Selected'            THEN NEW.status IN ('Offer_Made', 'Rejected')
605          ELSE FALSE   -- Offer_Made and Rejected are final
606      END;
607
608      IF NOT v_allowed THEN
609          RAISE EXCEPTION 'Application % cannot move from % to %',
610              NEW.application_id, OLD.status, NEW.status
611              USING ERRCODE = 'PL007';
612      END IF;
613
614      RETURN NEW;
615  END;
616  $$ LANGUAGE plpgsql;
617
618  CREATE TRIGGER trg_application_status_transition
619  BEFORE UPDATE OF status ON application
620  FOR EACH ROW
621  WHEN (OLD.status IS DISTINCT FROM NEW.status)
622  EXECUTE FUNCTION trg_fn_application_status_transition();
```

A.3 Stored procedure sp_close_drive

`db/schema.sql`, lines 682–712.

```
682  CREATE OR REPLACE PROCEDURE sp_close_drive(
683      p_drive_id        INTEGER,
684      INOUT p_rejected_count INTEGER DEFAULT NULL
685  )
686  LANGUAGE plpgsql
687  AS $$
688  DECLARE
689      v_status drive.status%TYPE;
690  BEGIN
691      -- Lock the drive row so two admins closing the same drive at once can't
692      -- both pass the "already closed?" check below.
693      SELECT status INTO v_status FROM drive WHERE drive_id = p_drive_id FOR UPDATE;
694
695      IF NOT FOUND THEN
696          RAISE EXCEPTION 'Drive % does not exist', p_drive_id USING ERRCODE = 'PL004';
697      END IF;
698
699      IF v_status IN ('Completed', 'Cancelled') THEN
700          RAISE EXCEPTION 'Drive % is already %', p_drive_id, v_status USING ERRCODE = 'PL005';
701      END IF;
702
703      UPDATE application
704         SET status = 'Rejected'
705       WHERE drive_id = p_drive_id
706         AND status IN ('Applied', 'Shortlisted', 'Interview_Scheduled');
707
708      GET DIAGNOSTICS p_rejected_count = ROW_COUNT;
709
710      UPDATE drive SET status = 'Completed' WHERE drive_id = p_drive_id;
711  END;
712  $$;
```

A.4 Window-function view with the month series (placement_timeline_view)

`db/schema.sql`, lines 756–773.

```
CREATE VIEW placement_timeline_view AS
WITH monthly AS (
    SELECT date_trunc('month', offer_date)::DATE AS month, COUNT(*) AS offers
    FROM offer
    GROUP BY date_trunc('month', offer_date)
),
all_months AS (
    SELECT gs::DATE AS month
    FROM (SELECT MIN(month) AS first_month, MAX(month) AS last_month FROM monthly) b,
         generate_series(b.first_month, b.last_month, INTERVAL '1 month') gs
)
SELECT
    m.month,
    COALESCE(x.offers, 0) AS offers_in_month,
    SUM(COALESCE(x.offers, 0)) OVER (ORDER BY m.month) AS cumulative_offers,
    COALESCE(x.offers, 0) - LAG(COALESCE(x.offers, 0)) OVER (ORDER BY m.month) AS change_from_prev_month
FROM all_months m
LEFT JOIN monthly x ON x.month = m.month;
```

A.5 The offer-acceptance transaction (backend)

`backend/services/offer_service.py`, lines 65–115.

```
65  def accept_offer(offer_id: int, requesting_user: dict) -> dict:
66      with db.get_connection() as conn:
67          cur = conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor)
68          try:
69              offer_row = _load_offer_owner(cur, offer_id)
70              if offer_row is None:
71                  raise OfferNotFound(f"Offer {offer_id} does not exist")
72              _authorize(requesting_user, offer_row["student_id"])
73              student_id = offer_row["student_id"]
74
75              # --- Statement 1: flip the Offer itself -----------------------
76              # WHERE status='Pending' + RETURNING doubles as an atomic guard:
77              # if two requests race to accept/decline the same offer, only
78              # the first UPDATE finds a matching row — the second gets back
79              # no row and this function raises instead of silently
80              # double-processing the offer.
81              cur.execute(
82                  """
83                  UPDATE offer
84                  SET status = 'Accepted',
85                      joining_date = COALESCE(joining_date, offer_date + INTERVAL '60 days')
86                  WHERE offer_id = %s AND status = 'Pending'
87                  RETURNING offer_id
88                  """,
89                  (offer_id,),
90              )
91              if cur.fetchone() is None:
92                  raise OfferNotActionable(
93                      f"Offer {offer_id} is not Pending (already actioned or invalid)"
94                  )
95
96              # --- Statement 2: update Student status ------------------------
97              cur.execute(
98                  "UPDATE student SET placement_status = 'Placed' WHERE student_id = %s",
99                  (student_id,),
100             )
101
102             # --- Statement 3: append to the audit trail ---------------------
103             cur.execute(
104                 "INSERT INTO offer_history (offer_id, action, notes) VALUES (%s, 'Accepted', %s)",
105                 (offer_id, "Offer accepted by student"),
106             )
107
108             conn.commit()
109         except Exception:
110             conn.rollback()
111             raise
112         finally:
113             cur.close()
114
115     return {"offer_id": offer_id, "status": "Accepted"}
```

Appendix B Screenshots

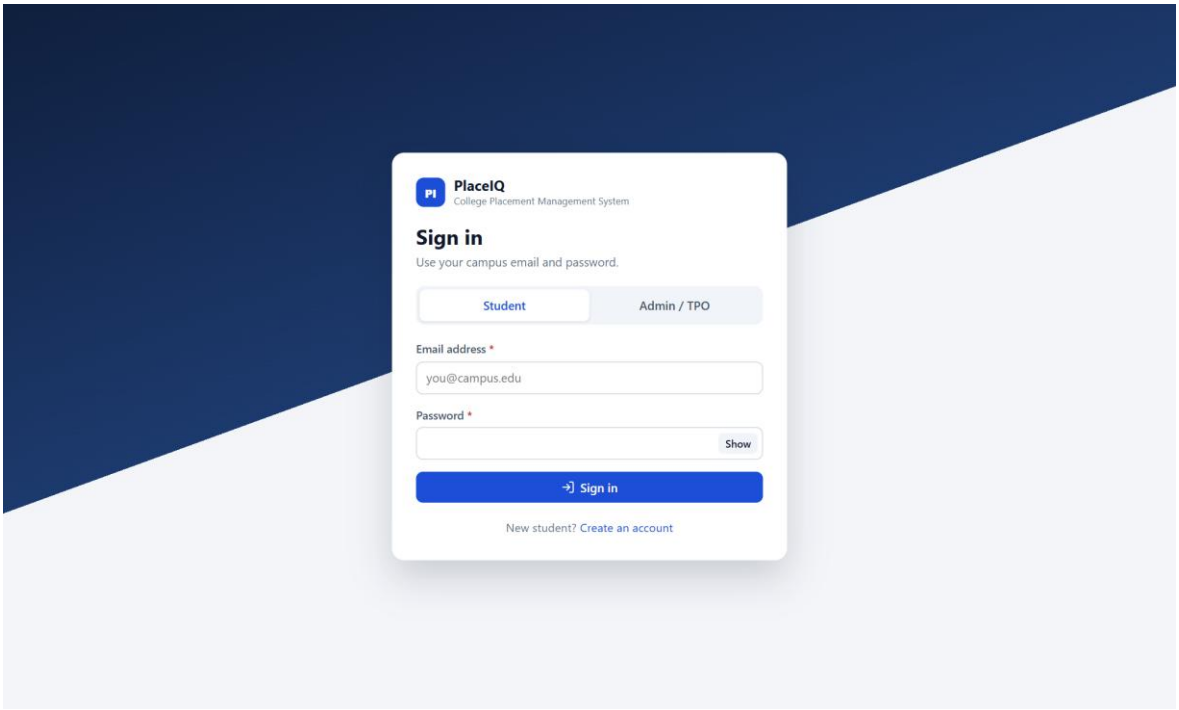


Figure 8: Sign-in page (student and placement-office roles)

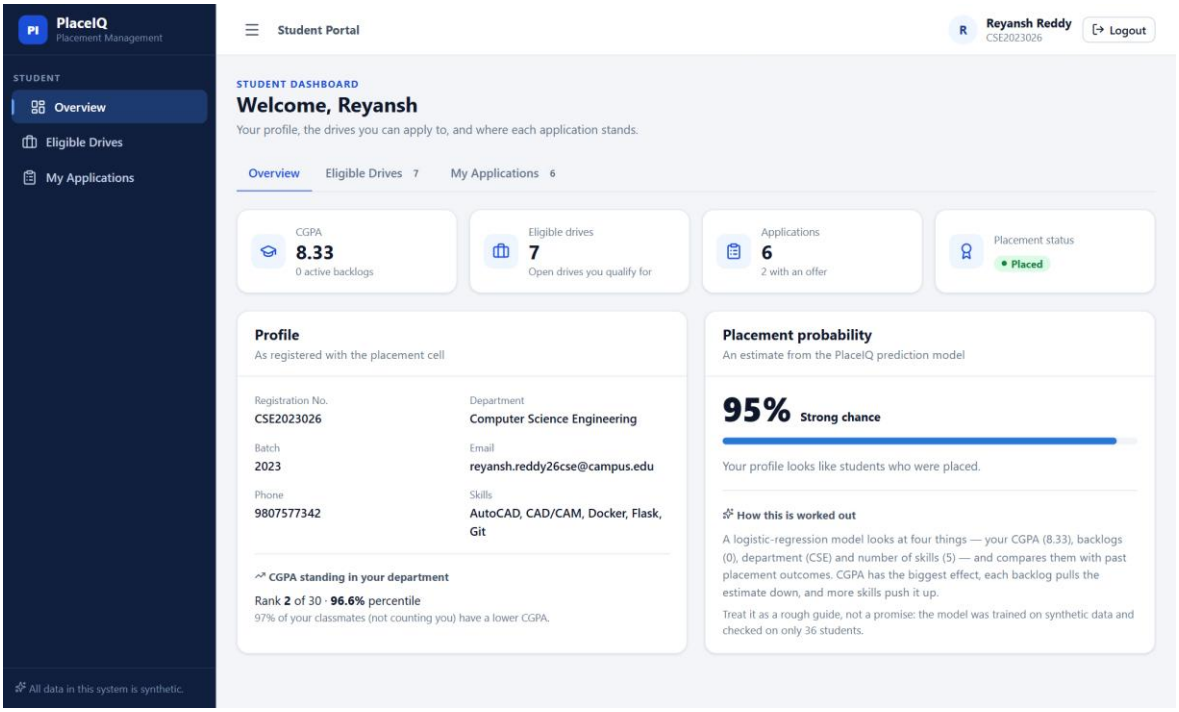


Figure 9: Student dashboard: profile, CGPA standing in the department, placement probability

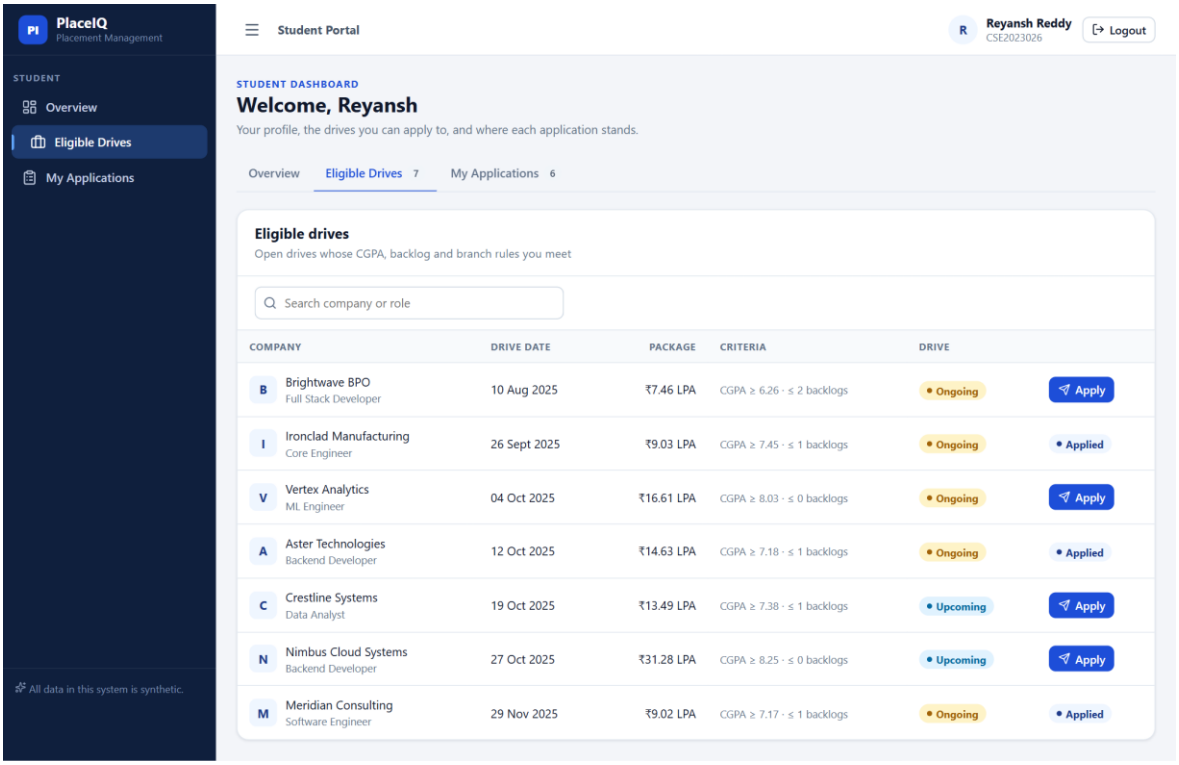


Figure 10: Eligible drives, from fn_get_eligible_drives

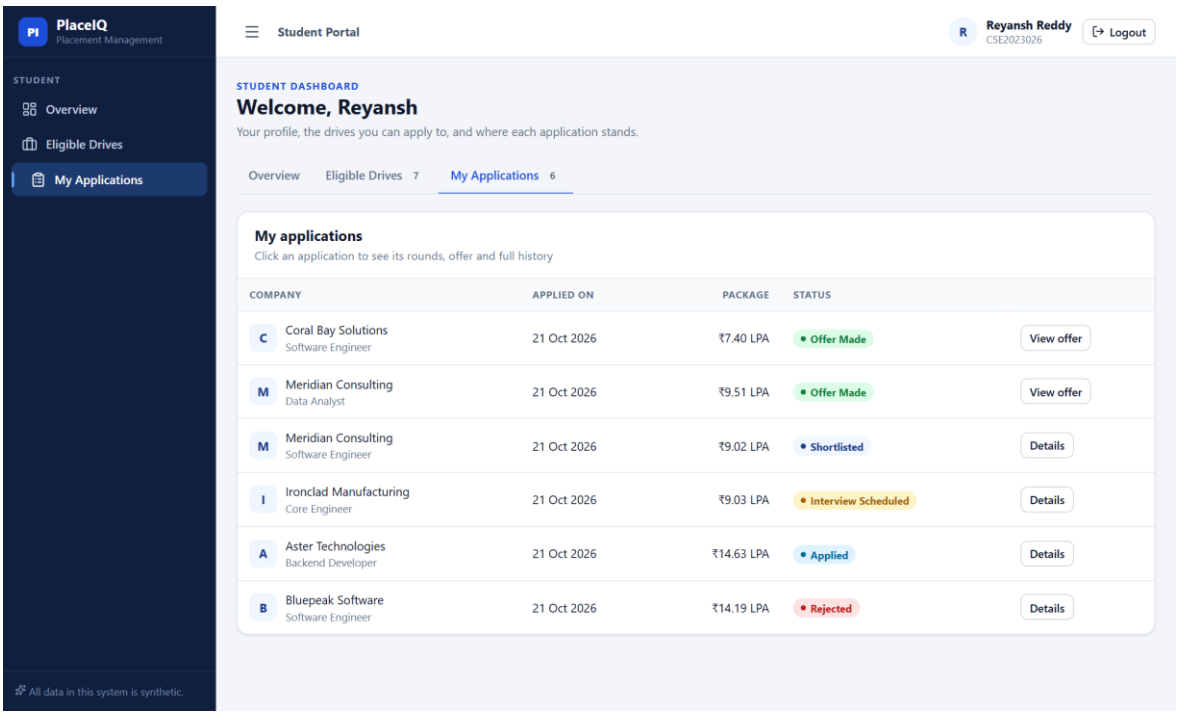


Figure 11: A student's applications at five different stages

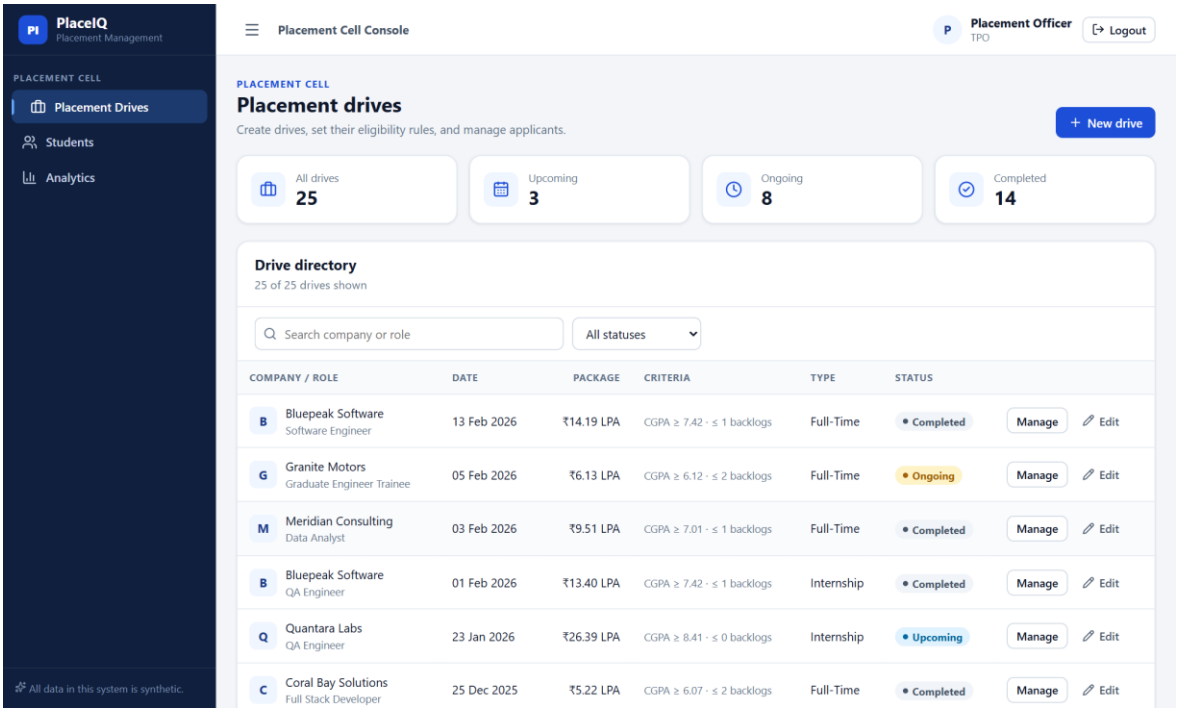


Figure 12: Placement office: list of drives

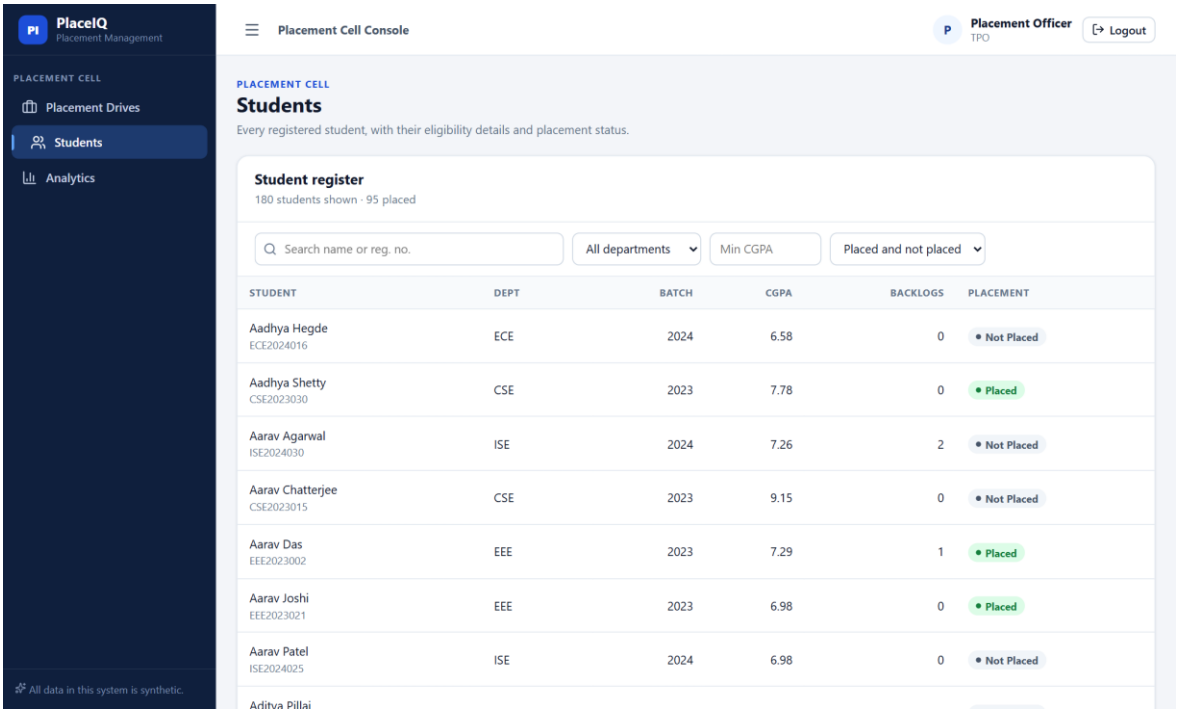


Figure 13: Placement office: student list

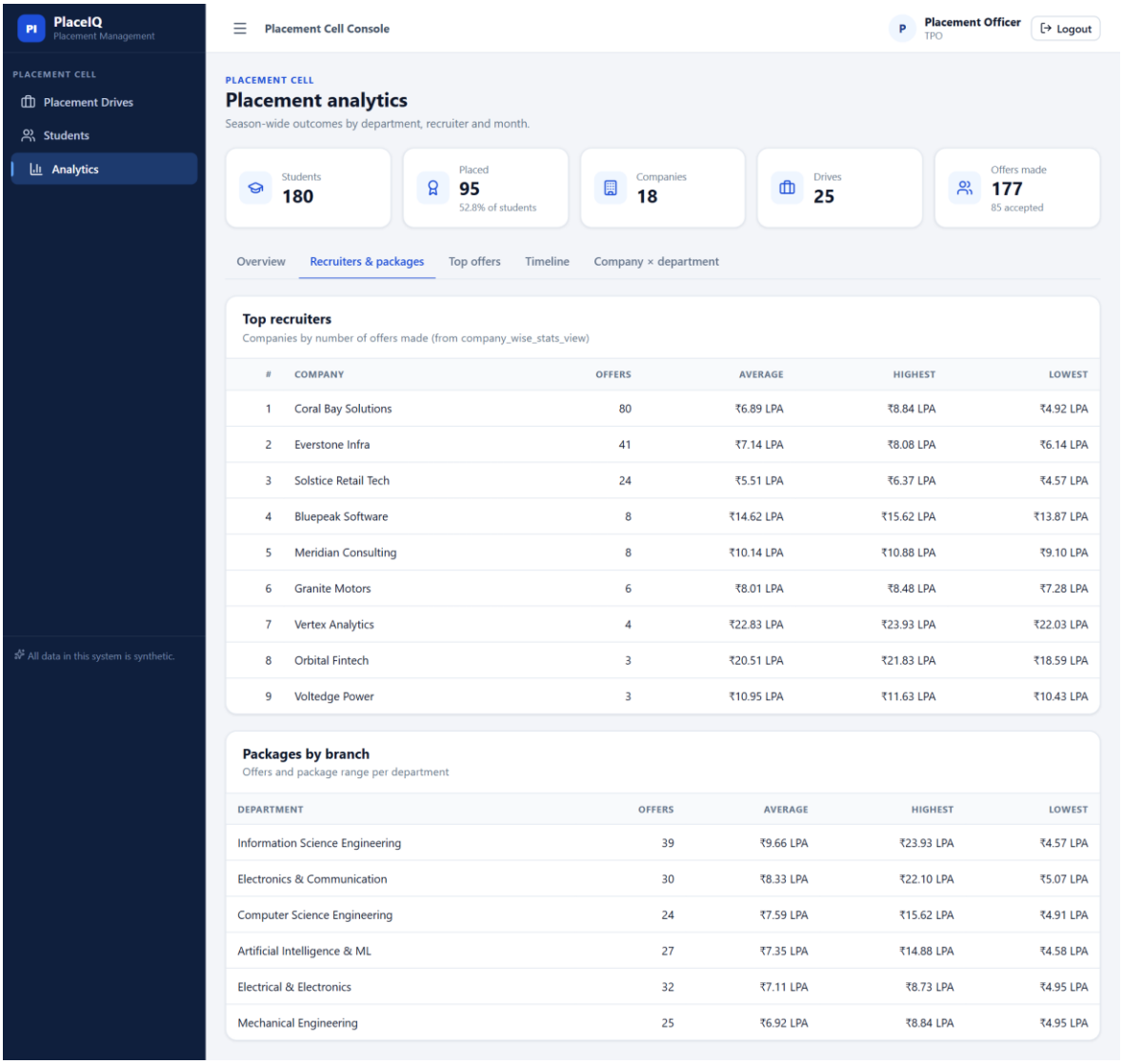


Figure 14: Analytics: recruiters and packages

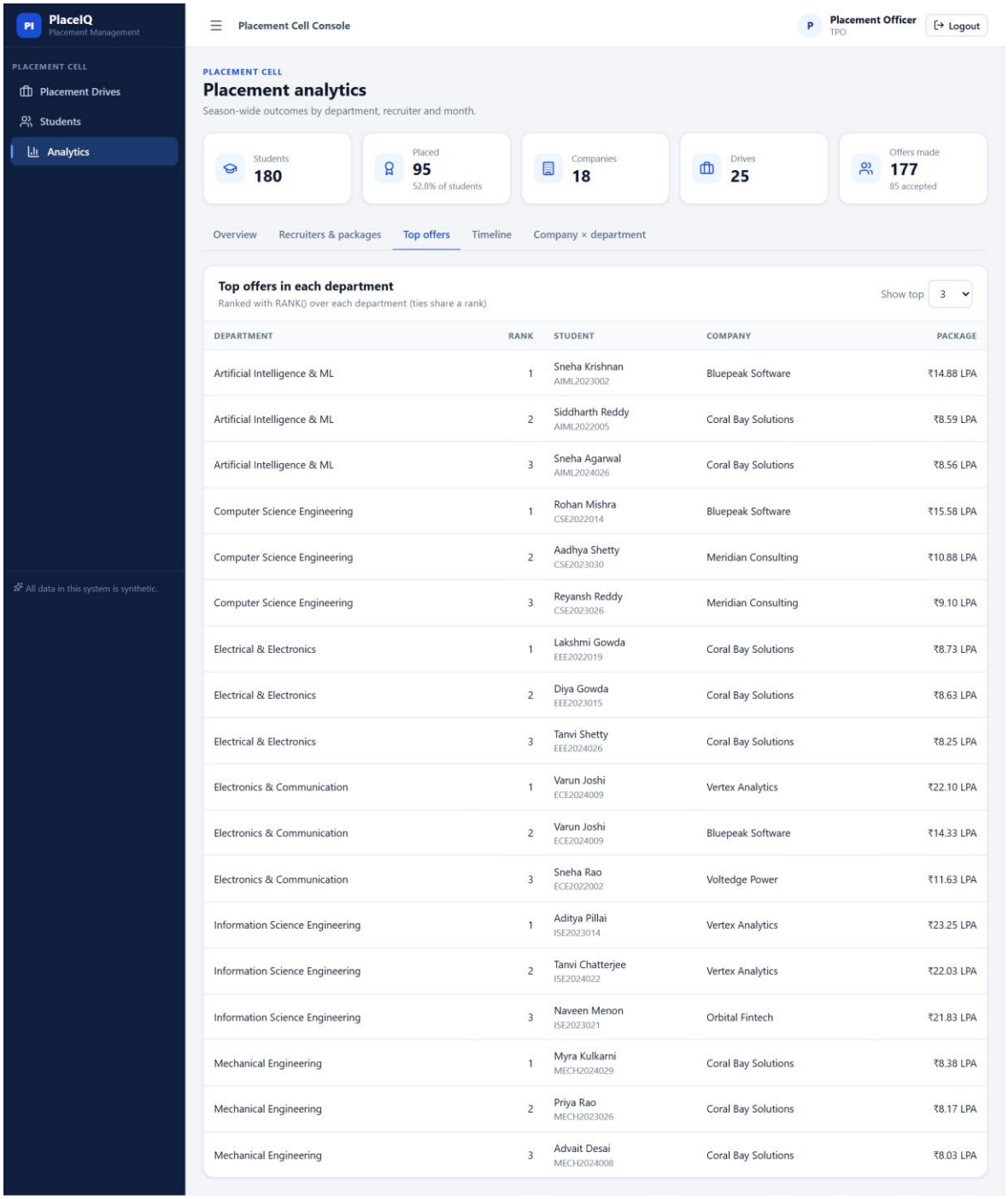


Figure 15: Analytics: top offers in each department (RANK)

Appendix C Evidence Files

The `docs/evidence/` folder of the repository holds the unedited output behind every result in this report. Its index (`docs/evidence/INDEX.md`) lists each file, the report section it supports and the script that regenerates it. Among them are the SQL*Plus transcript of the Oracle run, the full EXPLAIN ANALYZE plans of the benchmark, and the schema summary read from the PostgreSQL catalog.